



UNIVERSITÉ
AMADOU MAHTAR MBOW
DE DAKAR

Plus qu'une formation, un avenir à construire

UNIVERSITÉ
AMADOU MAHTAR MBOW
DE DAKAR

*Plus qu'une formation,
un avenir à construire.*

POLYTECH DIAMNIADIO,

MASTER EN INGÉNIERIE DES SYSTÈMES D'INFORMATION ET DE DONNÉES

UE : OUTILS MATHÉMATIQUES POUR L'INGÉNIEUR
EC : ANALYSE NUMÉRIQUE

Séquence 3 : SIMULATION NUMÉRIQUE D'EDO ET D'EDP

Dr. Elhadji Ibrahima Thiam

Sommaire

- ① Introduction
- ② Résolution et Simulation d'Équations Différentielles Ordinaires
 - EDO linéaire du premier ordre
 - EDO du second ordre
 - EDO avec condition initiale et condition limite
 - Méthodes de Runge Kutta pour la résolution des EDO
- ③ Calcul des coefficients $k_1, k_2, k_3, \dots$
- ④ Exemple pratique
- ⑤ Simulation d'EDP Stationnaire
 - Simulation de l'équation de Poisson
 - Simulation de l'équation de diffusion réaction
 - Applications
- ⑥ Simulation d'EDP Évolutive
 - Simulation numérique de l'équation de transport
 - Simulation Numérique de l'équation de la Chaleur
 - Applications

Introduction Générale

Ce document présente la résolution des équations différentielles ordinaires (EDO). Nous allons aborder différents exemples, y compris des EDO linéaires du premier et du second ordre, ainsi que des EDO non homogènes, des EDO à coefficients non constants et l'application de conditions initiales et limites.

- 2 Résolution et Simulation d'Équations Différentielles Ordinaires
 - EDO linéaire du premier ordre
 - EDO du second ordre
 - EDO avec condition initiale et condition limite
 - Méthodes de Runge Kutta pour la résolution des EDO

Exemple 1 : EDO linéaire du premier ordre

Considérons l'équation différentielle :

$$\frac{dy}{dx} + y = 0$$

La solution générale est obtenue par séparation des variables et intégration :

$$\frac{dy}{y} = -dx$$

$$\int \frac{1}{y} dy = \int -1 dx$$

$$\ln |y| = -x + C$$

$$y = Ce^{-x}$$

La solution est donc :

$$y = C_1 e^{-x}$$

Exemple 2 : EDO linéaire avec second membre

Considérons l'équation différentielle :

$$\frac{dy}{dx} + y = \sin(x)$$

Nous appliquons la méthode du facteur intégrant : $\mu(x) = e^{\int 1 dx} = e^x$

Multipliant l'équation par $\mu(x)$, nous obtenons :

$$e^x \frac{dy}{dx} + e^x y = e^x \sin(x)$$

Le membre de gauche devient une dérivée exacte :

$$\frac{d}{dx}(e^x y) = e^x \sin(x)$$

Intégrons des deux côtés :

$$e^x y = \int e^x \sin(x) dx$$

La solution de l'intégrale est :

$$e^x y = \frac{e^x}{2}(\sin(x) - \cos(x)) + C$$

Divisant par e^x , nous obtenons la solution :

$$y = \frac{1}{2}(\sin(x) - \cos(x)) + Ce^{-x}$$

2 Résolution et Simulation d'Équations Différentielles Ordinaires

- EDO linéaire du premier ordre
- EDO du second ordre
- EDO avec condition initiale et condition limite
- Méthodes de Runge Kutta pour la résolution des EDO

Exemple 3 : EDO du second ordre homogène

Considérons l'équation différentielle :

$$\frac{d^2y}{dx^2} - 3\frac{dy}{dx} + 2y = 0$$

L'équation caractéristique est :

$$r^2 - 3r + 2 = 0$$

Les racines de cette équation sont :

$$r_1 = 1 \quad \text{et} \quad r_2 = 2$$

La solution générale est donc :

$$y = C_1 e^{r_1 x} + C_2 e^{r_2 x}$$

$$y = C_1 e^x + C_2 e^{2x}$$

La solution est donc :

$$y = C_1 e^x + C_2 e^{2x}$$

Exemple 4 : EDO non homogène du second ordre

Considérons l'équation différentielle :

$$\frac{d^2 y}{dx^2} - y = e^x$$

La solution homogène est obtenue à partir de l'équation :

$$\frac{d^2 y}{dx^2} - y = 0$$

L'équation caractéristique est :

$$r^2 - 1 = 0 \quad \Rightarrow \quad r = \pm 1$$

La solution homogène est donc :

$$y_h = C_1 e^x + C_2 e^{-x}$$

Nous cherchons une solution particulière y_p de la forme : $y_p = A x e^x$

En substituant dans l'équation :

$$(2A + Ax)e^x - A x e^x = e^x \quad \Rightarrow \quad A = \frac{1}{2}$$

Ainsi, la solution particulière est : $y_p = \frac{x}{2} e^x$

La solution générale est donc :

$$y = C_1 e^x + C_2 e^{-x} + \frac{x}{2} e^x$$

2 Résolution et Simulation d'Équations Différentielles Ordinaires

- EDO linéaire du premier ordre
- EDO du second ordre
- EDO avec condition initiale et condition limite
- Méthodes de Runge Kutta pour la résolution des EDO

Exemple 5 : EDO avec condition initiale

Considérons l'équation différentielle suivante :

$$\frac{dy}{dx} - 2y = 0$$

Avec la condition initiale :

$$y(0) = 1$$

Méthode utilisée :

Séparation des variables :

$$\frac{dy}{y} = 2dx$$

Intégration des deux côtés :

$$\ln |y| = 2x + C$$

Exponentiation pour obtenir :

$$y = Ce^{2x}$$

Appliquer la condition initiale $y(0) = 1$:

$$1 = Ce^0 \Rightarrow C = 1$$

$$y = e^{2x}$$

Exemple 6 : EDO avec condition initiale et second membre

Considérons l'équation suivante :

$$\frac{dy}{dx} + y = \cos(x), \quad y(0) = 2$$

Méthode utilisée :

Calcul du facteur intégrant : $\mu(x) = e^{\int 1 dx} = e^x$

Multiplier l'équation par $\mu(x)$:

$$e^x \frac{dy}{dx} + e^x y = e^x \cos(x)$$

$$\frac{d}{dx}(e^x y) = e^x \cos(x)$$

Intégrer :

$$e^x y = \int e^x \cos(x) dx = \frac{e^x}{2}(\sin(x) + \cos(x)) + C$$

Appliquer la condition initiale : $2 = \frac{1}{2}(\sin(0) + \cos(0)) + C \Rightarrow C = \frac{3}{2}$

$$y = \frac{1}{2}(\sin(x) + \cos(x)) + \frac{3}{2}e^{-x}$$

Exemple 7 : EDO avec conditions aux limites

Considérons l'équation suivante :

$$\frac{d^2 y}{dx^2} - y = \sin(x), \quad y(0) = 0 \quad \text{et} \quad y(\pi) = 1$$

Méthode utilisée :

Résolution de l'équation homogène associée :

$$y_h = C_1 e^x + C_2 e^{-x}$$

Recherche d'une solution particulière :

$$y_p = A \cos(x) + B \sin(x)$$

Appliquer les conditions aux limites :

$$y(0) = 0 \quad \Rightarrow \quad C_1 + C_2 = 0$$

$$y(\pi) = 1 \quad \Rightarrow \quad C_1 e^\pi - C_2 e^{-\pi} = 1$$

Résolution :

$$C_1 = \frac{1}{e^\pi - e^{-\pi}}, \quad C_2 = -\frac{1}{e^\pi - e^{-\pi}}$$

Solution particulière :

$$y(x) = \frac{1}{e^\pi - e^{-\pi}} (e^x - e^{-x}) - \frac{1}{2} \sin(x)$$

Exemple 8 : EDO avec coefficients non constants

Considérons l'équation suivante :

$$x^2 \frac{d^2 y}{dx^2} + x \frac{dy}{dx} - y = 0$$

Méthode utilisée :

Supposer une solution de la forme :

$$y = x^r$$

Résoudre l'équation caractéristique :

$$r^2 - 2r + 1 = 0 \quad \Rightarrow \quad r = 1 \quad (\text{multiplicité double})$$

Solution générale :

$$y = C_1 x + C_2 x \ln(x)$$

Exemple 9 : EDO avec coefficients non constants

Considérons l'équation suivante :

$$x \frac{dy}{dx} + 2y = \sin(x)$$

Méthode : Variation des paramètres.

Divisons l'équation par x (en supposant $x \neq 0$) :

$$\frac{dy}{dx} + \frac{2}{x}y = \frac{\sin(x)}{x}$$

L'équation obtenue est une équation linéaire du premier ordre de la forme :

$$\frac{dy}{dx} + P(x)y = Q(x)$$

où $P(x) = \frac{2}{x}$ et $Q(x) = \frac{\sin(x)}{x}$. Le facteur intégrant $\mu(x)$ est donné par :

$$\mu(x) = e^{\int P(x) dx} = e^{2 \ln(x)} = x^2$$

Multiplions l'équation par $\mu(x) = x^2$ pour rendre le membre de gauche exact :

$$x^2 \frac{dy}{dx} + 2xy = x \sin(x)$$

Ce qui peut être réécrit comme une dérivée :

$$\frac{d}{dx}(x^2 y) = x \sin(x)$$

Intégrons les deux côtés par rapport à x :

$$\int \frac{d}{dx}(x^2 y) dx = \int x \sin(x) dx$$

L'intégrale de gauche est triviale :

$$x^2 y = \int x \sin(x) dx$$

Calculons cette dernière par intégration par parties :

$$\int x \sin(x) dx = -x \cos(x) + \int \cos(x) dx = -x \cos(x) + \sin(x)$$

Nous isolons y en divisant par x^2 :

$$y = \frac{-x \cos(x) + \sin(x) + C}{x^2}$$

La solution générale de l'équation différentielle est donc :

$$y(x) = \frac{-x \cos(x) + \sin(x)}{x^2} + \frac{C}{x^2}$$

- 2 Résolution et Simulation d'Équations Différentielles Ordinaires
 - EDO linéaire du premier ordre
 - EDO du second ordre
 - EDO avec condition initiale et condition limite
 - Méthodes de Runge Kutta pour la résolution des EDO

Introduction aux méthodes de Runge-Kutta

Les méthodes de Runge-Kutta sont des techniques itératives utilisées pour résoudre des équations différentielles ordinaires (EDO) du type :

$$\frac{dy}{dt} = f(t, y)$$

Ces méthodes permettent d'approximer la solution de $y(t)$ à partir d'une condition initiale y_0 .

Les méthodes de Runge-Kutta peuvent être classées en fonction de leur ordre, c'est-à-dire le nombre d'étapes utilisées pour effectuer l'approximation.

Méthode de Runge-Kutta d'ordre 4 (RK4)

La méthode de Runge-Kutta d'ordre 4 (RK4) est l'une des plus populaires, offrant une bonne précision tout en étant relativement simple à implémenter.

L'idée est de calculer plusieurs estimations de la pente $f(t, y)$ à différents points intermédiaires dans l'intervalle $[t_n, t_{n+1}]$. Ces estimations sont combinées pour obtenir une approximation précise de la solution.

Les étapes de la méthode RK4 sont :

Calcul de $k_1 = h \cdot f(t_n, y_n)$

Calcul de $k_2 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right)$

Calcul de $k_3 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right)$

Calcul de $k_4 = h \cdot f(t_n + h, y_n + k_3)$

Finalement, la solution à t_{n+1} est donnée par la combinaison pondérée des k -s :

$$y_{n+1} = y_n + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

Méthode de Runge-Kutta d'ordre supérieur

Pour obtenir des méthodes de Runge-Kutta d'ordre supérieur (par exemple, d'ordre 5 ou 6), le nombre d'étapes intermédiaires (et donc de coefficients $k_1, k_2, k_3, \dots$) augmente. Ces méthodes permettent d'obtenir des approximations plus précises de la solution.

Chaque méthode ajoute des termes supplémentaires pour améliorer la précision de l'approximation. La forme générale des coefficients suit la même logique que pour la méthode RK4, mais elle implique la résolution de systèmes d'équations plus complexes.

Les méthodes d'ordre supérieur peuvent avoir des coefficients supplémentaires et plus de points intermédiaires pour chaque étape.

Calcul des coefficients $k_1, k_2, k_3, \dots$

Les coefficients $k_1, k_2, \dots$ dans une méthode de Runge-Kutta sont obtenus par un processus qui utilise :

- 1) ****Développement en série de Taylor**** : Le développement en série de Taylor est utilisé pour approximer la solution $y(t_{n+1})$ à partir des évaluations successives de $f(t, y)$.
- 2) ****Consistance de la méthode**** : Les coefficients sont ajustés pour assurer que la méthode soit consistante et qu'elle atteigne l'ordre de précision voulu (par exemple, ordre 4 ou 5).
- 3) ****Méthode de Butcher**** : La méthode de Butcher est une technique formelle utilisée pour organiser les coefficients dans un tableau systématique, facilitant ainsi la résolution des équations algébriques nécessaires pour obtenir les coefficients.

Calcul des coefficients pour une méthode d'ordre 5

Pour une méthode de Runge-Kutta d'ordre 5, le calcul des coefficients k_1, k_2, k_3, k_4, k_5 est similaire à la méthode d'ordre 4, mais avec plus de points d'évaluation et de termes intermédiaires.

Par exemple, pour une méthode d'ordre 5, les coefficients sont calculés ainsi :

$$k_1 = h \cdot f(t_n, y_n)$$

$$k_2 = h \cdot f\left(t_n + \frac{h}{4}, y_n + \frac{k_1}{4}\right)$$

$$k_3 = h \cdot f\left(t_n + \frac{h}{4}, y_n + \frac{k_2}{4}\right)$$

$$k_4 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_3}{2} - \frac{k_2}{2}\right)$$

$$k_5 = h \cdot f\left(t_n + \frac{3h}{4}, y_n + \frac{3}{8}k_1 + \frac{3}{8}k_4\right)$$

La solution à t_{n+1} est ensuite donnée par une combinaison pondérée de ces coefficients.

La méthode de Butcher permet d'organiser systématiquement les coefficients requis pour ces méthodes, facilitant la conception et l'analyse de leurs propriétés. La méthode de Butcher est une formalisation puissante pour développer et analyser les méthodes de Runge-Kutta :

- Simplifie le calcul et la mise en oeuvre des coefficients k_i .

- Permet la conception de méthodes d'ordre supérieur.

- Outil essentiel dans les simulations numériques modernes.

Structure du tableau de Butcher

Le tableau de Butcher s'organise comme suit :

c_1	a_{11}	a_{12}	$\dots$	a_{1s}
c_2	a_{21}	a_{22}	$\dots$	a_{2s}
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
c_s	a_{s1}	a_{s2}	$\dots$	a_{ss}
	b_1	b_2	$\dots$	b_s

c_i : Points intermédiaires d'évaluation.

a_{ij} : Coefficients pour les calculs intermédiaires.

b_i : Poids pour combiner les k_i .

Principe des méthodes de Runge-Kutta

Les méthodes de Runge-Kutta cherchent à approximer la solution $y(t)$ d'une EDO :

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

1) Calcul des coefficients k_i :

$$k_i = h \cdot f \left(t_n + c_i h, y_n + \sum_{j=1}^{i-1} a_{ij} k_j \right), \quad i = 1, \dots, s$$

2) Combinaison des coefficients :

$$y_{n+1} = y_n + \sum_{i=1}^s b_i k_i$$

Exemple : Méthode de Runge-Kutta d'ordre 4 (RK4)

Le tableau de Butcher pour RK4 :

0	0	0	0	0
$\frac{1}{2}$	$\frac{1}{2}$	0	0	0
$\frac{1}{2}$	0	$\frac{1}{2}$	0	0
1	0	0	1	0
	$\frac{1}{6}$	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{6}$

Formules de calcul :

$$k_1 = h \cdot f(t_n, y_n)$$

$$k_2 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right)$$

$$k_3 = h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right)$$

$$k_4 = h \cdot f(t_n + h, y_n + k_3)$$

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Exemple pratique : $\frac{dy}{dt} = -2y$

Données : $f(t, y) = -2y$, $y(0) = 1$, $h = 0.1$.

Calculs pour $t_1 = t_0 + h = 0.1$:

$$k_1 = 0.1 \cdot (-2 \cdot 1) = -0.2$$

$$k_2 = 0.1 \cdot (-2 \cdot (1 - 0.1)) = -0.18$$

$$k_3 = 0.1 \cdot (-2 \cdot (1 - 0.09)) = -0.182$$

$$k_4 = 0.1 \cdot (-2 \cdot (1 - 0.182)) = -0.1636$$

$$y_1 = 1 + \frac{1}{6}(-0.2 - 0.36 - 0.364 - 0.1636) = 0.8187$$

Avantages de la méthode de Butcher

Organisation systématique : Structure claire pour les coefficients.

Flexibilité : Adaptable à des méthodes explicites et implicites.

Extension naturelle : Simplifie la construction de méthodes d'ordre supérieur.

Sommaire

- 5 Simulation d'EDP Stationnaire
 - Simulation de l'équation de Poisson
 - Simulation de l'équation de diffusion réaction
 - Applications

Stabilité, consistance, convergence

Le choix d'une méthode de résolution est basé sur la géométrie du domaine de calcul, la complexité du modèle (le type d'EDP), mais surtout la stabilité et la convergence numérique.

- Stabilité : Une méthode, un schéma numérique est stable lorsque la différence entre la solution approchée et la solution exacte est bornée.
- Consistance : Lorsque les pas de discrétisation en temps comme en espace tendent vers zéros, une méthode ou schéma consistant donne une solution discrète qui tend vers la solution exacte.
- Convergence : Une méthode convergente fait tendre la solution discrète vers la solution des équations continues.

Convergence

Pour rappel, un schéma numérique est consistant lorsque la solution discrète approche la solution des équations continues quand la taille des éléments de discrétisation (h, τ) tendent vers zéro. La vérification la plus importante dans une simulation numérique est celle de l'ordre de convergence. On s'attend à ce que l'ordre de convergence calculé est bien celui attendu suivant les schémas numériques utilisés. Elle est importante dans un contexte où l'erreur de discrétisation dépend fortement des conditions de simulation.

Dans le cas où la solution exacte est connue, il convient de savoir si :

- La solution discrète converge vers la solution exacte lorsque les pas de discrétisations diminue.
- Quel est alors l'ordre de convergence en temps et en espace

Exemple 1 : EDP Elliptique

On considère le problème suivant :

$$\begin{cases} -\frac{\partial^2 u}{\partial x^2}(x) = f(x) & \text{dans }]0, 1[\\ u(0) = a, \quad u(1) = b. \end{cases} \quad (1)$$

Ce problème est la version stationnaire de l'équation de la chaleur pour un choix de terme source f indépendant du temps. L'équation discrétisée en considérant un maillage en $N + 1$ points x_i de l'intervalle $[0, 1]$ avec un pas de discrétisation constant de Δx est :

$$\frac{1}{\Delta x^2} (2u_i - u_{i+1} - u_{i-1}) = f_i, \quad i = 1, \dots, N, \quad u_0 = a, \quad u_N = b$$

Exemple 1 : EDP Elliptique

Il est très pratique d'utiliser la notation matricielle de ce schéma de différences finies :

$$\frac{1}{\Delta x^2} \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & -1 & 2 & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-1} \end{pmatrix} = \begin{pmatrix} f_1 + \frac{a}{\Delta x^2} \\ f_2 \\ \vdots \\ f_{N-1} + \frac{b}{\Delta x^2} \end{pmatrix}$$

Ce qui nous donne un vecteur solution U , une matrice A et un vecteur pour le terme source F

$$AU = \Delta x^2 F$$

Exemple 1 : EDP Elliptique

Avant la simulation de ce problème il est important de vérifier que le système $AU = \Delta x^2 F$ admet une unique solution, qu'une petite variation des données F n'entraîne qu'une légère variation de la solution numérique U , que pour $\Delta x \rightarrow 0$, la solution discrète U tend vers la solution $U_e = (u(x_1), \dots, u(x_{N-1}))^T$.

Le système $AU = \Delta x^2 F$ admet une unique solution puisque la matrice A est symétrique définie positive, donc inversible.

Nous allons montrer la consistance, l'ordre de convergence et la stabilité du schéma de différences pour l'équation elliptique 1D (1).

Consistance

En remplaçant dans le schéma numérique u_i par $u(x_i)$, où u est la solution exacte, l'erreur de consistance ou de troncature est

$$r_i = \frac{1}{\Delta x^2} (2u(x_i) - u(x_{i+1}) - u(x_{i-1})) - f(x_i) \text{ ou encore } R = AU_e - F.$$

Un schéma de différence est consistant si

$$\lim_{\Delta x \rightarrow 0} \max_{i=0, \dots, N+1} |r_i| = 0.$$

Le schéma est d'ordre p en espace si $\max_{i=0, \dots, N+1} |r_i| \leq C \Delta x^p$, C une constante indépendante de la solution u .

Si u est de classe $\mathcal{C}^4([0, 1])$, on a grâce au développement de Taylor d'ordre 4 :

$$\begin{aligned} u(x_{i+1}) &= u(x_i) + \Delta x u'(x_i) + \frac{\Delta x^2}{2} u''(x_i) + \frac{\Delta x^3}{6} u'''(x_i) + \frac{\Delta x^4}{24} u^{(4)}(\epsilon_i) \\ u(x_{i-1}) &= u(x_i) - \Delta x u'(x_i) + \frac{\Delta x^2}{2} u''(x_i) - \frac{\Delta x^3}{6} u'''(x_i) + \frac{\Delta x^4}{24} u^{(4)}(\mu_i) \end{aligned}$$

Consistance

$$r_i = -\frac{1}{\Delta x^2} \left(\Delta x^2 u''(x_i) + \frac{\Delta x^4}{24} u^{(4)}(\epsilon_i) + \frac{\Delta x^4}{24} u^{(4)}(\mu_i) \right) - f(x_i)$$

$$r_i = \frac{\Delta x^2}{24} \left(u^{(4)}(\epsilon_i) + u^{(4)}(\mu_i) \right)$$

D'où

$$|r_i| \leq \frac{\Delta x^2}{12} \max_{x \in [0,1]} |u^{(4)}(x)|$$

Ce qui prouve que le schéma est consistant et d'ordre 2 en espace.

Avant d'étudier la stabilité du schéma numérique proposé pour le problème (1), il est nécessaire de rappeler quelques fondamentaux sur la stabilité des méthodes numériques de résolution de systèmes linéaires.

Stabilité

On considère le problème d'inconnu u : Trouver u tel que

$$F(x, d) = 0 \quad (2)$$

Le problème (2) est bien posé, suivant la définition de Hadamard si :

- 1) La solution u existe et est unique,
- 2) dépend continûment des données d : $F(x + \delta x, d + \delta d) = 0$, δd , perturbation introduite sur les données, δx , perturbation induite par δd . Cette dépendance continue signifie que $\|\delta x\| \leq K(d)\|\delta d\|$, pour $\|\delta d\| \leq \epsilon$. Ceci exprime la stabilité. On définit alors le conditionnement relatif afin de la quantifier :

$$K(d) = \sup \left\{ \frac{\|\delta x\|}{\|x\|} \frac{\|d\|}{\|\delta d\|} \right\}$$

Stabilité

Si le problème (2) admet une unique solution, alors il existe une application G dite *Résolvante* telle que $x = G(d)$, $F(G(d), d) = 0$. Si G est différentiable on a :

$$K(d) \simeq \|G'(d)\| \frac{\|d\|}{\|G(d)\|}.$$

Considérons maintenant le système $AU = F$, résultant de la discrétisation de l'équation (1). A est inversible, on a alors

$$G(F) = A^{-1}F, \text{ et } G'(F) = A^{-1}, \text{ ainsi}$$

$$K(d) \simeq \|A^{-1}\| \frac{\|F\|}{\|A^{-1}F\|} = \|A^{-1}\| \frac{\|AU\|}{\|U\|} \leq \|A\| \|A^{-1}\| = K(A)$$

$K(A)$ est le conditionnement de la matrice A . Donc si le conditionnement de A est "petit", la résolution du système linéaire $AU = F$ est un problème stable par rapport aux perturbations du second membre F , plus le conditionnement de la matrice est grand, plus la solution du système linéaire est sensible aux perturbations des données. Ce conditionnement régle la stabilité des méthodes.

Stabilité

Le schéma numérique de l'équation (1) est dit stable pour la norme $\|\cdot\|$ si sa solution est continue par rapport aux données pour cette norme. Cette dernière dépend essentiellement de la fonction u solution du problème dans un espace fonctionnel bien défini. En d'autre terme, il existe une constante C telle que

$$\|U\| \leq C\|F\|$$

On a $\|U\|_\infty = \max_{i=0,\dots,N+1} \{|u_i|\}$, $\|A\|_\infty = \max_{i=0,\dots,N+1} \sum_{j=0}^{N+1} |a_{i,j}|$. Si U est la solution discrète du système $AU = F$, alors,

$$\|U\|_\infty \leq \|A^{-1}F\|_\infty \leq \|A^{-1}\|_\infty \|F\|_\infty.$$

Il suffit pour montrer la stabilité du schéma numérique, de démontrer que

$$\|A^{-1}\|_\infty \leq C$$

Stabilité

Sans rentrer dans les détail de la majoration de $\|A^{-1}\|_\infty$ on peut affirmer que pour le problème (1), on a

$$\|A^{-1}\|_\infty \leq \frac{1}{8}$$

Il n'est donc pas nécessaire d'avoir une condition sur le pas de discrétisation Δx pour la stabilité du schéma numérique. On dit que le schéma est *inconditionnellement stable*. La constante $\frac{1}{8}$ permet de déterminer l'erreur sur l'approximation du second membre $f(x_i)$.

Convergence

À partir de l'erreur de discrétisation on peut déterminer la convergence du schéma numérique, bien qu'un schéma consistant et stable est un schéma convergent. Le calcul de cette erreur permet de voir numériquement l'ordre de convergence du schéma, en codant $\|E\| = \|U_e - U\|$. La norme utilisée ici sera la norme $\|\cdot\|_\infty$. L'erreur de consistance R est liée à l'erreur de discrétisation E par la relation :

$$R = AU_e - F = AU_e - AU = A(U_e - U), \quad \text{et } U_e - U = A^{-1}R.$$

Donc la norme de l'erreur

$$\|E\|_\infty = \|A^{-1}R\|_\infty \leq \|A^{-1}\|_\infty \|R\|_\infty \leq \frac{1}{8} \frac{\Delta x^4}{12} \max_{x \in [0,1]} |u^{(4)}(x)|$$

$$\|E\|_\infty \leq \frac{\Delta x^2}{96} \|u^{(4)}(x)\|_\infty$$

Implémentation, Résultats

Tout ce qui précède concerne la modélisation numérique du problème étudié. Cette analyse mathématique est un préalable nécessaire à toute simulation. Il est présenté dans la suite, des implémentations de des vérifications de simulation sous Python. La transcription sous Scilab est demandée afin de comprendre et de s'exercer à l'adaptation de cette méthodologie.

Implémentation, Résultats

```
# =====
# Resolution de l Equation Elliptique
# -d^2u/dx=f avec x dans ]0,1[, avec
#     f(x) = sin(np.pi*x), et
#     la condition a la limite u(0)= u(1) = 0
# =====
# IMPORTAION DES LIBRAIRIES
import numpy as np
import matplotlib.pyplot as plt
import scipy.sparse as sp
from scipy.sparse.linalg import spsolve
from scipy.linalg import norm

### MAILLAGE DU DOMAINE

NX = 10;x = np.linspace(0.0,1,NX);dx = 1/(NX-1)

# DEFINITION DE LA FONCTION SOURCE F
def func1(x):
    return(np.sin(np.pi*x))
f = np.vectorize(func1)

# DEFINITION DE LA SOLUTION EXACTE
def func2(x):
    return((np.sin(np.pi*x))/(np.pi**2))
Uex = np.vectorize(func2)
```

Implémentation, Résultats

```
### Calcul de la Solution Discrete
```

```
def U_discret(x,NX,dx,f):  
    U = np.zeros((NX));  
    U[0] = 0; U[-1] = 0; #Conditions limites  
    F = (dx**2)*f(x);  
    F = F[1:NX-1]  
    val_diags = [-np.ones(NX-2),2*np.ones(NX-2),-np.ones(NX-2)]  
    positions = np.array([-1,0,1])  
    A = sp.dia_matrix((val_diags,positions), shape=(NX-2,NX-2))  
    Uint = spsolve(A,F); U[1:NX-1] = Uint  
    return(U)
```

```
### # AFFICHAGE DES SOLUTIONS DISCRETES ET EXACTES COMPAREES
```

```
U = U_discret(x,NX,dx,f)  
Uexact= Uex(x);  
fig, ax = plt.subplots(1)  
ax.plot(x, U, 'r-+',lw=2, label='Solution Numerique')  
ax.plot(x, Uexact,'b-o',lw=2, label='Solution Analytique')  
ax.set_xlabel('x') ;ax.set_ylabel('y') ;  
ax.set_xlim([0, 1])  
ax.set_ylim([0, 0.11])  
ax.set_title('Comparaison Solutions')  
ax.legend();ax.grid()
```

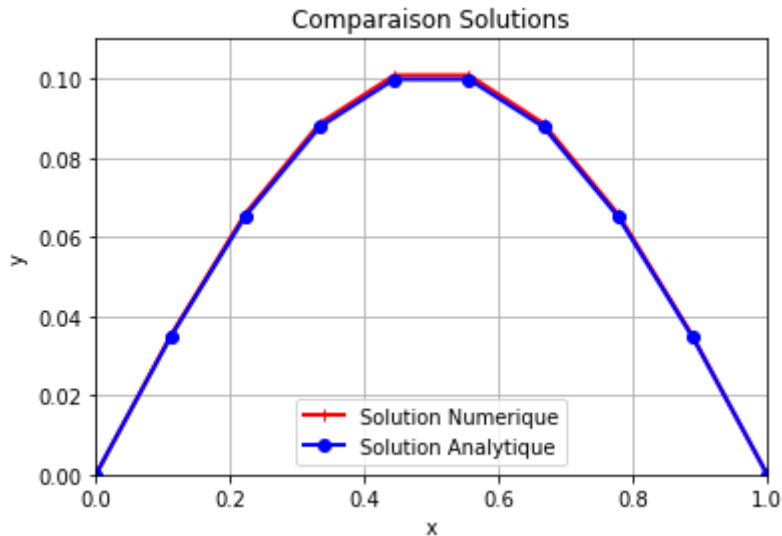
Implémentation, Résultats

```
#####
%% ETUDE DE LA CONSISTENCE, DE LA CONVERGENCE EN ESPACE DU SCHEMA
#####

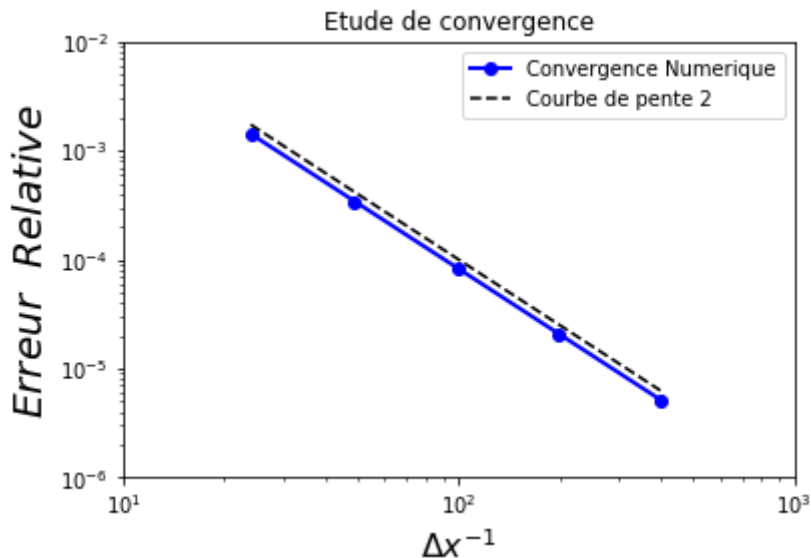
NX = np.array([25,50,100,200,400]); dx = 1/(NX-1); NN = NX.size;
ERRABS = np.zeros(NN);ERRLAT = np.zeros(NN);
for k in range(0,NN):
    x = np.linspace(0.0,1,NX[k]);
    UC = U_discret(x,NX[k],dx[k],f)
    Uexact= Uex(x);
    ERRABS[k] = max(abs(UC-Uexact))
    ERRLAT[k] = max(abs(UC-Uexact)) / max(abs(Uexact))
    print(norm((UC-Uexact)), " : NORME de l'erreur absolue")
P1 = np.log((ERRLAT[1] - ERRLAT[2])/(ERRLAT[2] - ERRLAT[3]))/np.log(dx[0]/dx[1])
print('Ordre de Convergence en erreur Relative: ',P1)
P2 = np.log((ERRABS[1] - ERRABS[2])/(ERRABS[2] - ERRABS[3]))/np.log(dx[0]/dx[1])
print('Ordre de Convergence en erreur absolue : ',P2)

fig, ax = plt.subplots(1)
ax.loglog(dx**-1, ERRLAT,'b-o',lw=2,label='Convergence Numerique')
ax.loglog(dx**-1,1*dx**2,'--k', label='Courbe de pente 2')
ax.set_xlabel('$\Delta x^{-1}$', fontsize=16);
ax.set_ylabel('$Erreur \sim Relative$', fontsize=20)
ax.set_xlim([10**1, 10**3])
ax.set_ylim([10**-6, 10**-2])
ax.set_title('Etude de convergence');ax.legend()
```

Graphiques



Graphiques



Sommaire

- 5 Simulation d'EDP Stationnaire
 - Simulation de l'équation de Poisson
 - Simulation de l'équation de diffusion réaction
 - Applications

Équation de diffusion réaction stationnaire

On considère maintenant l'équation diffusion réaction stationnaire :

$$\begin{cases} -u''(x) + c(x)u(x) = f(x) & \text{dans }]0, 1[\\ u(0) = \alpha, \quad u(1) = \beta. \end{cases} \quad (3)$$

Pour une fonction c positive, les termes $u''(x)$ et $c(x)u(x)$ représentent respectivement la diffusion et la réaction de $u(x)$.

Dans le cas où c est une constante, la solution analytique du problème (3) est connue. Par contre pour une fonction c positive, dans l'espace $L^\infty([0, 1])$ et $f \in L^2([0, 1])$, la formulation variationnelle nous assure de l'existence et de l'unicité de la solution. On ne dispose donc pas de formule simple donnant la solution. L'approximation du problème par un schéma de différences finies permet alors d'approcher cette solution.

Approximation

L'équation discrétisée en considérant un maillage en $N + 1$ points x_i de l'intervalle $[0, 1]$ avec un pas de discrétisation constant de $\Delta x = h$ est :

$$\frac{1}{h^2} (2u_i - u_{i+1} - u_{i-1}) + c_i u_i = f_i, \quad i = 1, \dots, N, \quad u_0 = \alpha, \quad u_N = \beta \quad (4)$$

Le problème s'écrit : $AU = F$ avec :

$$A = \left(\frac{1}{h^2} \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & -1 & 2 & -1 \\ 0 & \cdots & \cdots & -1 & 2 \end{pmatrix} + \begin{pmatrix} c_1 & 0 & 0 & \cdots & 0 \\ 0 & c_2 & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & c_{N-2} & 0 \\ 0 & \cdots & \cdots & 0 & c_{N-1} \end{pmatrix} \right),$$

$$U = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-1} \end{pmatrix}, \quad F = \begin{pmatrix} f_1 + \frac{\alpha}{h^2} \\ f_2 \\ \vdots \\ f_{N-1} + \frac{\beta}{h^2} \end{pmatrix}$$

Simulation Validation

Pas à pas, nous allons dérouler la simulation du problème (3). Pour $\alpha = 0$, $\beta = 0$, on s'intéresse d'abord au cas où la fonction c est une constante positive.

Il sera question de :

- 1) Donner la solution analytique pour $c = 1$ et $f(x) = x^2 \exp(x)$.
- 2) Comparer sous Scilab ou Python, en s'inspirant des codes de la première section, la solution approchée et la solution exacte.
- 3) Que ce passe t-il aux limites du domaine ?
- 4) Étudier théoriquement la consistance et la convergence du schéma numérique employé.
- 5) Vérifie numériquement l'ordre de convergence.

Simulation Validation

On pose maintenant $c(x) = x^2$ et $f(x) = \exp(2x)$. La solution analytique de (3), bien qu'elle existe, n'est pas explicitement fournie. Ce qui donne un intérêt très particulier à l'approximation (4). Il s'agit :

- 1) Construire la matrice A et le second membre F .
- 2) Donner sur graphique la solution discrète.
- 3) Peut-on trouver à partir de la solution discrète, la solution exacte du problème ? Si oui Comment ?

Sommaire

- 5 Simulation d'EDP Stationnaire
 - Simulation de l'équation de Poisson
 - Simulation de l'équation de diffusion réaction
 - Applications

Application 1 : Équation de diffusion réaction généralisée

On considère l'équation diffusion réaction stationnaire, où μ , γ sont des constantes positives :

$$\begin{cases} -\mu u''(x) + \gamma u(x) = 0 & \text{dans }]0, 1[\\ u(0) = 0, \quad u(1) = 1. \end{cases} \quad (5)$$

- 1) Donner la solution analytique de (5).
- 2) Écrire un programme simulant la solution de (5) et la comparer à la solution exacte, pour le même pas d'espace h et pour un choix de μ et γ , tel que :

a) $\frac{\gamma}{6\mu} \ll 1$

b) $\frac{\gamma}{6\mu} \gg 1$

Ce nombre est le Péclet qui mesure le rapport entre les phénomènes diffusifs et réactifs. S'il est $\gg 1$ c'est la réaction qui domine sur la diffusion, s'il est $\ll 1$ c'est la diffusion qui domine.

- 3) Pour $\frac{\gamma}{6\mu} \gg 1$ de l'ordre de 1000 par exemple, varier le pas d'espace h en prenant $\frac{h^2\gamma}{6\mu} \leq 1$ et $\frac{h^2\gamma}{6\mu} \geq 1$. Ce nombre est le Péclet local.

- 4) Quel critère de stabilité pouvez-vous en sortir ?

Application 2 : Équation de convection diffusion généralisée

On considère l'équation de convection diffusion stationnaire, où μ , γ sont positives :

$$\begin{cases} -\mu u''(x) + \gamma u'(x) = 0 & \text{dans }]0, 1[\\ u(0) = 0, \quad u(1) = 0. \end{cases} \quad (6)$$

- 1) Donner la solution analytique de (6).
- 2) Pour un choix de μ et γ tel que le nombre de Péclet, $\frac{\gamma}{2\mu} \gg 1$, par exemple 1000, comparer les schémas de différences finies :
 - a) $\frac{\mu}{h^2} (2u_i - u_{i+1} - u_{i-1}) + \frac{\gamma}{2h} (u_{i+1} - u_{i-1}) = 0$, $i = 1, \dots, N$, $u_0 = 0$, $u_{N+1} = 1$
 - b) $\frac{\mu}{h^2} (2u_i - u_{i+1} - u_{i-1}) + \frac{\gamma}{h} (u_i - u_{i-1}) = 0$, $i = 1, \dots, N$, $u_0 = 0$, $u_{N+1} = 1$,dans une simulation donnant deux solutions discrètes avec un pas d'espace h ("grand").
- 3) Lequel des schémas donne une solution oscillante.
- 4) Pour ce schéma, donnant une solution oscillante, comparer les solutions que donneraient une simulation avec $\frac{h\gamma}{2\mu} < 1$ et $\frac{h\gamma}{2\mu} > 1$. Ce nombre est le Péclet local.
- 5) Quel critère de stabilité pouvez-vous en sortir ?

Simulation d'EDP Hyperbolique et Parabolique

Cette séquence du cours traite de la validation et de la vérification de la modélisation numérique d'équations aux dérivées partielles hyperbolique et parabolique simples. Il s'agit de programmer, tester et valider des schémas de différences. On traitera des problèmes à solution analytique connue sur l'équation de transport et celle de la chaleur.

Sommaire

6 Simulation d'EDP Évolutive

- Simulation numérique de l'équation de transport
- Simulation Numérique de l'équation de la Chaleur
- Applications

L'équation de convection

On considère le modèle suivant :

$$\begin{cases} \frac{\partial u}{\partial t} + \alpha \frac{\partial u}{\partial x} = f & \text{dans }]0, 1] \times \mathbb{R}_+^* \\ u(0, t) = g(t) = \sin(10\pi t) & \text{sur } \mathbb{R}_+^* \\ u(x, 0) = u_0(x) = x & \text{dans } [0, 1] \end{cases} \quad (7)$$

Cette équation modélise le transport d'une quantité u dépendant de la position x et de l'instant t , sachant l'état initial et la vitesse de transport α . Lorsque la fonction f est identiquement nulle, on parle de transport libre.

Pour $f = 0$, $\alpha > 0$, la solution exacte de ce problème est donné par :

$$u(x, t) = \begin{cases} u_0(x - \alpha t) & \text{si } x - \alpha t > 0 \\ g(t - \frac{x}{\alpha}) & \text{si } x - \alpha t < 0 \end{cases}$$

Approximation numérique

Pour simuler ce modèle, on se place dans le domaine spatial $[0, 1]$ et temporel $[0, T_f]$, en posant

$$\begin{cases} x_i = i * \Delta x, & i \in \{0, \dots, N_X\} \\ t_n = n * \Delta t, & n \in \{0, \dots, N_T\} \end{cases}$$

On considère deux schémas de différences, le schéma décentré à gauche et le schéma de Lax Wendroff, en posant $\lambda = \alpha \frac{\Delta t}{\Delta x}$:

$$u_i^{n+1} = u_i^n - \lambda (u_i^n - u_{i-1}^n)$$

$$u_i^{n+1} = u_i^n - \frac{\lambda}{2} (u_{i+1}^n - u_{i-1}^n) + \frac{\lambda^2}{2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

Ces schémas seront comparés en simulation suivant leur ordre de convergence et leur efficacité à donner une meilleure approximation.

Ces schémas explicites peuvent être écrits sous la forme :

$$u_i^{n+1} = \mathfrak{D}(u_i^n) \tag{8}$$

avec $\mathfrak{D}$ une application linéaire.

Étude de consistance des schémas explicites

Nous allons étudier de façon générale la consistance sur les schémas explicites comme donnés ci-dessus. On se place dans le cadre où l'équation (7) est sans conditions limites ni second membre. En remplaçant dans le schéma numérique u_i^n par $u(x_i, t_n) = \tilde{u}_i^n$, où u est la solution exacte, l'erreur de consistance ou de troncature est :

$$r_i^{n+1} = \frac{1}{\Delta t} \left(\tilde{u}_i^{n+1} - \mathfrak{D}(\tilde{u}_i^n) \right).$$

Un schéma de différence du type (8) est consistant si

$$\lim_{\Delta t, \Delta x \rightarrow 0} \max_{i,n} |r_i^n| = 0.$$

Le schéma est d'ordre p en temps et d'ordre q en espace si

$$r_i^n(\Delta t, \Delta x) = \mathcal{O}(\Delta t^p, \Delta x^q).$$

Un schéma de différence du type (8) est convergent si

$$\lim_{\Delta t, \Delta x \rightarrow 0} \max_{i,n} |\tilde{u}_i^n - u_i^n| = 0$$

Consistance su schéma explicite décentré

Soit le schéma : $u_i^{n+1} = u_i^n - \lambda (u_i^n - u_{i-1}^n)$. Par un développement de Taylor de :

$$u(x_j, t^{n+1}) = u(x_j, t^n) + \Delta t \frac{\partial u(x_j, t^n)}{\partial t} + \frac{\Delta t^2}{2!} \frac{\partial^2 u(x_j, t^n)}{\partial t^2} + \mathcal{O}(\Delta t^3) \text{ et de}$$

$$u(x_{j-1}, t^n) = u(x_j, t^n) - \Delta x \frac{\partial u(x_j, t^n)}{\partial x} + \frac{\Delta x^2}{2!} \frac{\partial^2 u(x_j, t^n)}{\partial x^2} + \mathcal{O}(\Delta x^3),$$

en utilisant la notation $u(x_i, t_n) = \tilde{u}_i^n$, l'erreur de consistance est donnée par :

$$\begin{aligned} r_i^{n+1} &= \frac{1}{\Delta t} \left(\tilde{u}_i^n + \Delta t \frac{\partial \tilde{u}_i^n}{\partial t} + \frac{\Delta t^2}{2!} \frac{\partial^2 \tilde{u}_i^n}{\partial t^2} + \mathcal{O}(\Delta t^3) \right) \\ &- \frac{1}{\Delta t} \left(\tilde{u}_i^n - \lambda \left(\tilde{u}_i^n - \tilde{u}_{i-1}^n + \Delta x \frac{\partial \tilde{u}_i^n}{\partial x} - \frac{\Delta x^2}{2!} \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} + \mathcal{O}(\Delta x^3) \right) \right) \\ &= \frac{\partial \tilde{u}_i^n}{\partial t} + \frac{\Delta t}{2} \frac{\partial^2 \tilde{u}_i^n}{\partial t^2} + \mathcal{O}(\Delta t^2) + \frac{\lambda}{\Delta t} \left(\Delta x \frac{\partial \tilde{u}_i^n}{\partial x} - \frac{\Delta x^2}{2} \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} - \mathcal{O}(\Delta x^3) \right) \\ &= \frac{\partial \tilde{u}_i^n}{\partial t} + \alpha \frac{\partial \tilde{u}_i^n}{\partial x} + \frac{\Delta t}{2} \frac{\partial^2 \tilde{u}_i^n}{\partial t^2} - \frac{\alpha \Delta x}{2} \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} + \mathcal{O} \left(\Delta t^2, \frac{\Delta x^3}{\Delta t} \right). \end{aligned}$$

Puisque $\frac{\partial \tilde{u}_i^n}{\partial t} + \alpha \frac{\partial \tilde{u}_i^n}{\partial x} = 0$ alors

$$r_i^{n+1} = \frac{\Delta t}{2} \frac{\partial^2 \tilde{u}_i^n}{\partial t^2} - \frac{\alpha \Delta x}{2} \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} + \mathcal{O} \left(\Delta t^2, \frac{\Delta x^3}{\Delta t} \right)$$

Consistance su schéma explicite décentré

de plus $\left(\frac{\partial^2 \tilde{u}_i^n}{\partial t^2} = -\alpha \frac{\partial}{\partial t} \left(\frac{\partial \tilde{u}_i^n}{\partial x} \right), \frac{\partial}{\partial x} \left(\frac{\partial \tilde{u}_i^n}{\partial t} \right) = -\alpha \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} \right) \Rightarrow \frac{\partial^2 \tilde{u}_i^n}{\partial t^2} - \alpha^2 \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} = 0,$

Alors

$$\begin{aligned} r_i^{n+1} &= \frac{\alpha^2 \Delta t}{2} \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} - \frac{\alpha \Delta x}{2} \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} + \mathcal{O} \left(\Delta t^2, \frac{\Delta x^3}{\Delta t} \right) \\ &= \left(\frac{\alpha^2 \Delta t}{2} - \frac{\alpha \Delta x}{2} \right) \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} + \mathcal{O} \left(\Delta t^2, \frac{\Delta x^3}{\Delta t} \right) \\ &= \left(\frac{\alpha^2 \Delta t \Delta x}{2 \Delta x} - \frac{\alpha \Delta x^2}{2 \Delta x} \right) \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} + \mathcal{O} (\Delta t^2, \Delta x^2) \\ r_i^{n+1} &= (\lambda - 1) \frac{\alpha \Delta x}{2} \frac{\partial^2 \tilde{u}_i^n}{\partial x^2} + \mathcal{O} (\Delta t^2, \Delta x^2) \end{aligned}$$

- Si $\lambda = \alpha \frac{\Delta t}{\Delta x} = 1$ le schéma est au moins d'ordre 2 en espace et 2 en temps.
- Si $\lambda = \alpha \frac{\Delta t}{\Delta x} \neq 1$ le schéma est d'ordre 1 en espace et 2 en temps.

Ceci est valable lorsque le rapport $\frac{\Delta t}{\Delta x}$ est constant.

Application

Étudier la consistance des schémas pour l'équation d'advection :

$$\frac{\partial u}{\partial t} + \alpha \frac{\partial u}{\partial x} = 0,$$

suivants :

- Schéma explicite centré

$$u_i^{n+1} = u_i^n - \lambda (u_{i+1}^n - u_{i-1}^n)$$

- Schéma explicite décentré droite (lorsque $\alpha < 0$)

$$u_i^{n+1} = u_i^n - \lambda (u_{i+1}^n - u_i^n)$$

- Schéma de Lax-Friedrichs

$$u_i^{n+1} = \frac{1}{2} (u_{i-1}^n + u_{i+1}^n) + \frac{\lambda}{2} (u_{i+1}^n - u_{i-1}^n)$$

Stabilité des schémas explicites

Un schéma numérique approchant un problème hyperbolique (qu'il soit linéaire ou non linéaire) est stable si pour tout temps $t^n = n\Delta t \leq T_f$, il existe une constante C_t (dépendant ou non de T_f) telles que :

$$\|u(x, t^n)\|_{\Delta} \leq C_t \|u(x, t^0)\|_{\Delta}$$

pour tout $\Delta t, \Delta x, 0 \leq \Delta t, \Delta x \leq \delta_0, \delta_0 > 0$.

La norme $\|\cdot\|_{\Delta}$ peut être une version discrète des normes dans $L^1(\mathbb{R})$, $L^2(\mathbb{R})$ ou $L^\infty(\mathbb{R})$. Par exemple :

$$\|u(x, t^n)\|_{\Delta}^2 = \left(\sum_{i \in \mathbb{Z}} \Delta x |u(x_i, t^n)|^2 \right)^{\frac{1}{2}} = \sqrt{\Delta x} \|u(x, t^n)\|_{l^2(\mathbb{Z})}.$$

Une condition nécessaire et suffisante de stabilité de schémas explicites du type (8) est donnée par Courant, Friedrichs et Lewy, appelée "*Condition CFL*" :

$$\Delta t \leq \frac{\Delta x}{\alpha}, \quad \text{ou } \lambda \leq 1.$$

Stabilité des schémas explicites

Il est montré que les schémas :

- Explicite centré est inconditionnellement instable

$$u_i^{n+1} = u_i^n - \lambda (u_{i+1}^n - u_{i-1}^n)$$

- Explicite décentré gauche (lorsque $\alpha > 0$) est stable à condition $\lambda \leq 1$.

$$u_i^{n+1} = u_i^n - \lambda (u_i^n - u_{i-1}^n)$$

- Explicite décentré droite (lorsque $\alpha < 0$) est stable à condition que $\lambda \leq 1$.

$$u_i^{n+1} = u_i^n - \lambda (u_{i+1}^n - u_i^n)$$

- Schéma de Lax-Friedrichs est stable à condition que $\lambda \leq 1$

$$u_i^{n+1} = \frac{1}{2} (u_{i-1}^n + u_{i+1}^n) + \frac{\lambda}{2} (u_{i+1}^n - u_{i-1}^n)$$

- Law-Wendroff est stable à condition que $\lambda \leq 1$

$$u_i^{n+1} = u_i^n - \frac{\lambda}{2} (u_{i+1}^n - u_{i-1}^n) + \frac{\lambda^2}{2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

Validation des schémas

Nous allons chercher la solution discrète associée au problème (7) avec les différents schémas.

```
import numpy as np
import matplotlib.pyplot as plt
from numpy import linalg as la
#Definition du domaine de Calcul
a = 0; b = 1;
# Vitesse d advection
alpha = 10
# MAILLAGE DU DOMAINE
# Nombre de points; points de discretisation; pas d espace
NX = 100;x = np.linspace(0.0,1,NX);dx = 1/(NX-1)
# Choix du Pas de Temps (c*dt)/dx = CFL <=1
CFL = 0.5; dt = CFL*dx/alpha;
# Parametres numerique
Lambda = (alpha*dt)/dx;
NT = int(np.floor(1/dt))+1 ; t = 0;
# Condition Initiale
def Func1(x):
    return (x)
U0 = np.vectorize(Func1)
# Condition Limite
def Ulim(t):
    return(np.sin(10*np.pi*t))
# Solution Exacte
def Uex(a,b,alpha,x,t,U0,Ulim):
    return (np.where((x-alpha*t)>=a,U0(x-alpha*t),Ulim(t- (x-a)/alpha)))
```

Schéma explicite centré

```
%% Calcul de la Solution Discrete avec le schema explicite centre
def U_discret1(x,dt,Lambda,NX,NT,U0,Ulim):
    #Initialisation de la Solution Discrete
    U = np.zeros((NX,NT))
    U[:,0] = U0(x); t = 0; N = NX-1
    # Boucle en Temps de Calcul de la Solution Discrete
    for n in range(0,NT-1):
        U[1:N,n+1] = U[1:N,n] - (Lambda/2)*( U[2:N+1,n] - U[0:N-1,n] )
        t = t+dt;
        U[0,n+1] = Ulim(t) # Condition limite
        U[N,n+1] = U[N,n] - (Lambda/2)*( U[N-1,n] - U[N-2,n] )
    return(U)
```

```
%% AFFICHAGE DES SOLUTIONS DISCRETES ET EXACTES COMPAREES
```

```
UU = U_discret1(x,dt,Lambda,NX,NT,U0,Ulim);
t = 0;
for n in range(0,NT):
    Uexact = Uex(a,b,alpha,x,t,U0,Ulim)
    if (n%10 == 0):
        fig, ax = plt.subplots(1)
        labe = "Solution discrete a t = %1.2f" %(n * dt);
        ax.plot(x, UU[:,n],'b-o',lw=2, label = labe)
        ax.plot(x, Uexact,'r-+',lw=2, label = 'Solution exacte')
        plt.xlabel('$x$', fontsize=16);plt.ylabel('$U(x,t)$', fontsize=20)
        plt.title('Equation d advection 1D'); plt.ylim([-1, 1]); plt.xlim([0, 1]);
        plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.05), shadow=True, ncol=2);
    t = t+dt
```

Schéma explicite centré

En faisant varier Δx et Δt on s'aperçoit que le schéma explicite centré ne donne jamais une solution discrète en accord avec la solution exacte. Ce qui illustre le fait qu'il soit inconditionnellement instable.

Schéma décentré

La solution discrète donnée par le schéma décentré gauche est en accord avec la solution exacte du problème pour les paramètres numériques choisis.

Application

- 1) Coder la fonction $Udiscret2(x, dt, Lambda, NX, NT, U0, Ulim)$, donnant la solution par le schéma décentré gauche.
- 2) Changer la vitesse d'advection en $\alpha = 2, 3, 5, 8$ et regarder les solutions discrètes données comparées aux solutions exactes.
- 3) Quel constat faites-vous, quel est ce phénomène? Peut-on ajuster les paramètres numériques de sorte à pallier à ce problème?
- 4) Coder la fonction $Udiscret2(x, dt, Lambda, NX, NT, U0, Ulim)$, donnant la solution par le schéma de Lax-wendroff.
- 5) En changeant la vitesse d'advection, le schéma de Lax-wendroff donne t-il le même phénomène?

Stabilité numérique du schéma décentré

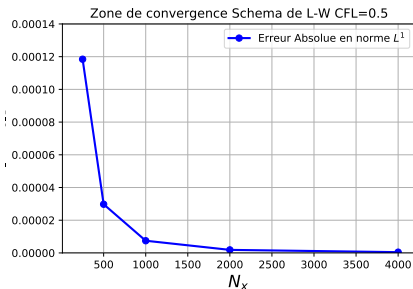
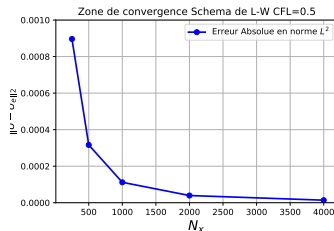
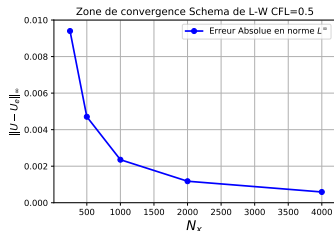
Il est question d'étudier la condition CFL du problème. La variable $CFL = (\alpha \Delta t) / \Delta x$ prendra trois valeurs (>1 , 1 , <1) afin de tester la condition $CFL \leq 1$ et de voir la stabilité du schéma pour des pas de temps différents. Le résultat pour le schéma décentré est donné :

Implémentation de l'étude de stabilité

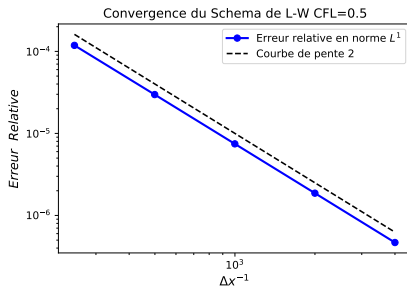
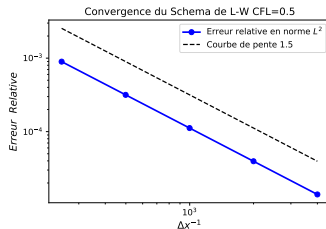
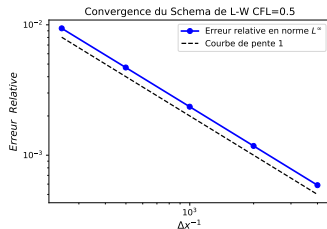
```
###=====
# ETUDE DE LA STABILITE DU SCHEMA
#=====
NX = 100; x = np.linspace(0.0,1,NX);dx = 1/(NX-1)
CFLT = np.array([1.5]);
NC = CFLT.size
dt = CFLT*dx/alpha; NTT = np.zeros(NC);
Lambda = (alpha*dt)/dx;
for k in range(0,NC):
    FF = np.floor(1/dt[k])
    NTT = int(FF) + 1;
    US = U_discret2(x,dt[k],Lambda[k],NX, NTT,U0,Ulim); t = 0;
    for n in range(0,NTT):
        Uexact = Uex(a,b,alpha,x,t,U0,Ulim);
        if (n%10 == 0):
            fig, ax = plt.subplots(1); labe = "Solution discrete CFL = %1.2f" %CFLT[k];
            ax.plot(x, US[:,n],'b-o',lw=2, label = labe)
            ax.plot(x, Uexact,'r-',lw=2, label = 'Solution exacte t = %1.2f' %(n * dt[k]))
            plt.xlabel('$x$', fontsize=16);plt.ylabel('$U(x,t)$', fontsize=12)
            plt.title('Equation d advection 1D Schema decentre');
            plt.ylim([-1.1, 1.1]); plt.xlim([-0.1, 1.1])
            plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.05), shadow=True, ncol=2);
            filename = "ADVECT44/Figure-" +str(CFLT[k]) + "_" + str(int(n/10))+".pdf"
            plt.savefig(filename);
    t = t+dt[k]
```

Stabilité numérique du schéma de Lax-Wendroff

Zone asymptotique de Convergence



Ordre de Convergence



Implémentation

```
%%=====
# ETUDE DE LA CONSISTENCE, DE LA CONVERGENCE EN ESPACE DU SCHEMA
#=====

NX = np.array([250,500,1000,2000,4000]); dx = 1/(NX-1)
CFLT = 0.5;
N = NX.size; NTC = np.zeros(N); dt = (CFLT*dx/alpha);
ERRABS = np.zeros(N); ERRLAT = ERRABS; ERRABS2 =ERRABS;ERRLAT2 = ERRABS;

for k in range(0,N):
    IN = int(np.floor(1/dt[k]))+1 ;
    NTC = int(IN)+1
    x = np.linspace(0.0,1,NX[k]);
    Lambda = (alpha*dt[k])/dx[k];
    UC = U_discret2(x,dt[k],Lambda,NX[k],NTC,U0,Ulim);
    UC2 = U_discret3(x,dt[k],Lambda,NX[k],NTC,U0,Ulim);
    t = 0
    for n in range(0,NTC):
        Uexact = Uex(a,b,alpha,x,t,U0,Ulim)
        t = t+dt[k]

    ERRABS[k] = la.norm(UC[:,n]-Uexact,np.inf);
    ERRLAT[k] = la.norm(UC[:,n]-Uexact,np.inf) / la.norm(Uexact,np.inf)
    ERRABS2[k] = la.norm(UC2[:,n]-Uexact,np.inf);
    ERRLAT2[k] = la.norm(UC2[:,n]-Uexact,np.inf) / la.norm(Uexact,np.inf)

print(); print(ERRABS[k]," : Erreur Absolue Schema decentre")
print(); print(ERRABS2[k]," : Erreur Absolue Schema de L W")
```

Implémentation

```
###
plt.figure(1)
plt.plot(NX,ERRABS, 'b-o',lw=2,label='Erreur Absolue en norme  $L^\infty$ ')
plt.xlabel('$N_x$', fontsize=16);plt.ylabel('$\Vert U - U_e \Vert_\infty$', fontsize=12)
plt.title("Zone de convergence Schema Decentre CFL=0.5");
plt.xlim([10, 4200]);plt.ylim([0., 0.01]);plt.grid(); plt.legend()
plt.figure(2)
plt.plot(NX,ERRABS2, 'b-o',lw=2,label='Erreur Absolue en norme  $L^\infty$ ')
plt.xlabel('$N_x$', fontsize=16);plt.ylabel('$\Vert U - U_e \Vert_\infty$', fontsize=12)
plt.title("Zone de convergence Schema de L-W CFL=0.5");
plt.xlim([10, 4200]);plt.ylim([0., 0.01]);plt.grid(); plt.legend()
###
P1 = np.log((ERRLAT[2] - ERRLAT[3])/(ERRLAT[3] - ERRLAT[4]))/np.log(dx[0]/dx[1])
print('Ordre de Convergence en erreur Relative Sc Dec: ',P1)
P2 = np.log((ERRABS[2] - ERRABS[3])/(ERRABS[3] - ERRABS[4]))/np.log(dx[0]/dx[1])
print('Ordre de Convergence en erreur absolue Sc Dec : ',P2)
P3 = np.log((ERRLAT2[2] - ERRLAT2[3])/(ERRLAT2[3] - ERRLAT2[4]))/np.log(dx[0]/dx[1])
print('Ordre de Convergence en erreur Relative Sc LW: ',P1)
P4 = np.log((ERRABS2[2] - ERRABS2[3])/(ERRABS2[3] - ERRABS2[4]))/np.log(dx[0]/dx[1])
print('Ordre de Convergence en erreur absolue Sc LW : ',P2)
fig, ax = plt.subplots(1)
ax.loglog(dx**-1, ERRLAT,'b-o',lw=2,label='Erreur relative en norme  $L^\infty$ ')
ax.loglog(dx**-1,2*dx**(1),'--k', label='Courbe de pente 1')
ax.set_xlabel('$\Delta x^{-1}$',fontsize=12);ax.set_ylabel('$Erreur-Relative$')
ax.set_title('Convergence du Schema decentre CFL=0.5');ax.legend()
fig2, ax2 = plt.subplots(1)
ax2.loglog(dx**-1, ERRLAT,'b-o',lw=2,label='Erreur relative en norme  $L^\infty$ ')
ax2.loglog(dx**-1,2*dx**(1),'--k', label='Courbe de pente 1')
ax2.set_xlabel('$\Delta x^{-1}$',fontsize=12);ax2.set_ylabel('$Erreur-Relative$')
ax2.set_title('Convergence du Schema de L-W CFL=0.5');ax2.legend()
```

Application

- 1) En se basant sur la séquence 1 du cours, expliquer l'étude de convergence effectuée.
- 2) Justifier et expliciter les normes utilisées avec *numpy.linalg* ?
- 3) Coder ces normes ?

Sommaire

6 Simulation d'EDP Évolutive

- Simulation numérique de l'équation de transport
- Simulation Numérique de l'équation de la Chaleur
- Applications

Équation de la chaleur

On considère l'équation de la chaleur dans un domaine borné $\Omega = [0, 1]$:

$$\begin{cases} \frac{\partial u}{\partial t} - \nu \Delta u = 0 & \text{dans } \Omega \times [0, T_f] \\ u = 0 & \text{sur } \partial\Omega \times [0, T_f] \\ u(t = 0) = u_0(x) & \text{dans } \Omega \end{cases} \quad (9)$$

Nous allons tester et comparer deux schémas de différences finies ici :

- un schéma explicite : $\frac{u_i^{n+1} - u_i^n}{\Delta t} - \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} = 0$
- un schéma implicite : $\frac{u_i^{n+1} - u_i^n}{\Delta t} - \nu \frac{u_{i+1}^{n+1} - 2u_i^{n+1} + u_{i-1}^{n+1}}{\Delta x^2} = 0$

En considérant les vecteurs $U^n = (u_1^n, \dots, u_{N-1}^n)$ et U^{n+1} , ces deux schémas donnent les systèmes linéaires :

- un schéma explicite : $U^{n+1} = A_1 U^n$
- un schéma implicite : $U^n = A_2 U^{n+1}$

Approximation numérique

En posant $\gamma = \frac{\nu \Delta t}{\Delta x^2}$

$$A_1 = \begin{pmatrix} 1 - 2\gamma & \gamma & 0 & \cdots & 0 \\ \gamma & 1 - 2\gamma & \gamma & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \gamma & 1 - 2\gamma & \gamma \\ 0 & 0 & 0 & \gamma & 1 - 2\gamma \end{pmatrix}$$

$$A_2 = \begin{pmatrix} 1 + 2\gamma & -\gamma & 0 & \cdots & 0 \\ -\gamma & 1 + 2\gamma & -\gamma & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & -\gamma & 1 + 2\gamma & -\gamma \\ 0 & 0 & 0 & -\gamma & 1 + 2\gamma \end{pmatrix}$$

Étude de la consistance du schéma explicite

Le schéma explicite pour l'équation de la chaleur est donnée :

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} - \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} = 0.$$

Il peut être réécrit pour $U^k = (u_1^k, \dots, u_{N-1}^k)$ sous la forme :

$$\frac{U^{n+1} - U^n}{\Delta t} + \nu A_h U^n = 0, \quad A_h = \frac{1}{\Delta x^2} \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{pmatrix}.$$

En remplaçant dans le schéma numérique u_i^n par $u(x_i, t_n) = \tilde{u}_i^n$, où u est la solution exacte, l'erreur de consistance ou de troncature avec $\tilde{u}_i^n = u(x_i, t^n)$ est :

$$r_i^n = \frac{1}{\Delta t} \left(\tilde{u}_i^{n+1} - \tilde{u}_i^n \right) + \nu A_h \tilde{u}_i^n.$$

Étude de la Consistance

Cette erreur s'écrit encore comme :

$$r_i^n = K_i - L_i, \quad K_i = \frac{1}{\Delta t} (\tilde{u}_i^{n+1} - \tilde{u}_i^n) - \frac{\partial \tilde{u}_i^n}{\partial t}, \quad L_i = \nu \frac{\tilde{u}_{i+1}^n - 2\tilde{u}_i^n + \tilde{u}_{i-1}^n}{\Delta x^2} - \frac{\partial^2 \tilde{u}_i^n}{\partial x^2}$$

Par un développement de Taylor relativement à la variable t , on a :

$$\tilde{u}_i^{n+1} = \tilde{u}_i^n + \Delta t \frac{\partial \tilde{u}_i^n}{\partial t} + \frac{\Delta t^2}{2} \frac{\partial^2 u(x_i, \tau)}{\partial t^2}, \quad \tau \in]t^n, t^{n+1}[.$$

$$\text{Ainsi } K_i = \frac{\Delta t^2}{2} \frac{\partial^2 u(x_i, \tau)}{\partial t^2}.$$

D'après la séquence 2, sur l'étude de l'erreur de troncature pour l'équation de diffusion stationnaire, on obtient :

$$L_i = \frac{\Delta x^2}{12} \frac{\partial^4 u(\epsilon, t^n)}{\partial x^4}, \quad \epsilon \in]x_i, x_{i+1}[.$$

Par suite, si la solution u du problème (9) de classe $\mathcal{C}^2$ relativement à la variable t et de classe $\mathcal{C}^2$ relativement à la variable x , alors, il existe une constante $C > 0$:

$$\|r_i^n\|_\infty \leq C (\Delta t + \Delta x^2)$$

Le schéma est consistant, d'ordre 1 en temps et d'ordre 2 en espace. De plus, si l'on considère la norme $\|\cdot\|_\Delta$, il est aussi consistant avec les mêmes ordres. On montre de même que le schéma implicite est consistant avec les mêmes ordres.

Étude de la Stabilité

On étudie d'abord la stabilité en norme $\|\cdot\|$, le schéma explicite écrit sous la forme :

$$U^{n+1} = (Id - \nu \Delta t A_h) U^n. \quad (10)$$

On en déduit que

$$U^k = (Id - \nu \Delta t A_h)^k U^0 = B^k U^0, \quad k = 1, 2, \dots.$$

La matrice B est appelée *matrice d'itération* ou *matrice d'amplification*. Un schéma du type (10) est stable s'il existe une constante $C_{T_f} > 0$ indépendant de Δt et Δx :

$$\max_{n, n\Delta t \leq T_f} \|B\| \leq C_{T_f}.$$

En d'autres termes, le schéma du type (10) est stable si $\lim_{k \rightarrow \infty} U^k = 0$, d'où, la convergence de la matrice B^k . On sait aussi que

$$\lim_{k \rightarrow \infty} B^k = 0 \Leftrightarrow \rho(B) = \max |(valeur propre de B)| \leq 1.$$

Les valeurs propres de B sont données par :

$$\lambda_k = 1 - \frac{4\nu \Delta t}{\Delta x^2} \sin^2 \left(\frac{k\pi}{2(N+1)} \right), \quad k = 1, \dots, N.$$

Étude de la Stabilité

De façon générale, pour une matrice d'ordre N , $A = \begin{pmatrix} a & b & 0 & \cdots & 0 \\ b & a & b & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & & a & b \\ 0 & 0 & 0 & b & a \end{pmatrix}$, les valeurs propres sont données par : $\lambda_k = a + 2b \cos\left(\frac{k\pi}{N+1}\right)$, $k = 1, \dots, N$. Donc le schéma explicite pour l'équation (9) est stable à la condition que

$$\rho(B) \leq 1 \Leftrightarrow \Delta t \leq \frac{\Delta x^2}{2\nu}.$$

Pour le schéma implicite, la matrice d'itération $B = (Id + \nu\Delta t A_h)^{-1}$. Les valeurs propres de cette matrice sont :

$$\lambda_k = \frac{1}{1 + \frac{4\nu\Delta t}{\Delta x^2} \sin^2\left(\frac{k\pi}{2(N+1)}\right)}, \quad k = 1, \dots, N$$

On a $0 \leq \lambda_k \leq 1$, $\forall k$. Donc le rayon spectral de la matrice B est inférieur à 1. Le schéma implicite est donc inconditionnellement stable pour tout $\Delta x > 0$, $\Delta t > 0$.

Application 4 : Simulation Numérique

Il s'agit dans ce cas test de vérifier dans un premier temps que le schéma explicite est stable à la condition $\gamma \leq 0.5$; que le schéma implicite est inconditionnellement stable. Les ordres de convergence seront calculés et comparés à ceux théoriques.

Nous prendrons comme donnée initiale : $u_0(x) = \sin(2\pi x)$ et la solution exacte correspondante est donnée pour $\nu = 0.5$:

$$u(x, t) = (\sin(2\pi x)) * (\exp(-4\pi^2 \nu t))$$

Il sera donné les implémentations des fonctions calculant les solutions par le schéma explicite et le schéma implicite. Il est demandé, en se basant sur ce qui précède, d'écrire les codes Scilab et Python correspondant aux études de stabilité et de convergence en espace de ces schémas. *Augmenter ou diminuer le coefficient de diffusion ν et commenter le comportement de la solution temporelle.*

Calcul de la solution discrète : Scilab

```
function U = U_DISCRETE1(x,dx,dt,Alpha,NX,NT,U0)
    U = zeros(NX,NT)
    U(:,1) = SolInitial(x);
    U(1,:) = 0; U(NX,:) = 0;
    N = NX-1; gam = (Alpha*dt)/(dx^2)
    A = diag((1-2*gam)*ones(N-1))+...
        diag(gam*ones(N-2),1)+diag(gam*ones(N-2),-1)
    for n=1:NT
        U(2:N-1,n+1) = A*U(2:N-1,n)
    end
endfunction

function U = U_DISCRETE2(x,dx,dt,Alpha,NX,NT,U0)
    U = zeros(NX,NT)
    U(:,1) = SolInitial(x);
    U(1,:) = 0; U(NX,:) = 0;
    N = NX-1; gam = (Alpha*dt)/(dx^2)
    A = diag((1+2*gam)*ones(N-1))+...
        diag(-gam*ones(N-2),1)+diag(-gam*ones(N-2),-1)
    for n=1:NT
        U(2:N-1,n+1) = inv(A)*U(2:N-1,n)
    end
endfunction
```


Calcul de la solution discrète : Python

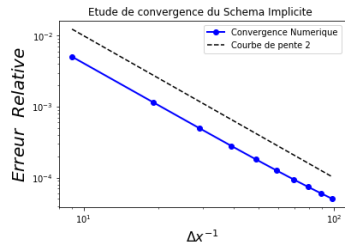
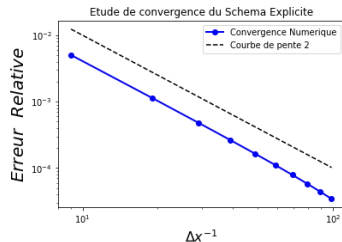
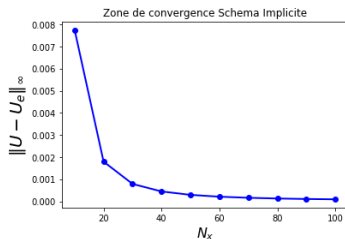
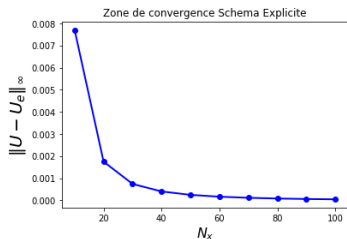
```
%% Calcul de la Solution Discrete
```

```
def U_discret1(x,dx,dt,Alpha,NX,NT,U0):
    U = np.zeros((NX,NT))
    U[:,0] = U0(x);
    U[0,:] = 0; U[-1,:] = 0;
    N = NX-1; gamma = (Alpha*dt)/(dx**2)
    A = np.diagflat((1-2*gamma)*np.ones(N-1),k=0) \
    +np.diagflat(gamma*np.ones(N-2),k=1)+np.diagflat(gamma*np.ones(N-2),k=-1)
    for n in range(0,NT-1):
        U[1:N,n+1] = np.dot(A,U[1:N,n])
    return(U)

def U_discret2(x,dx,dt,Alpha,NX,NT,U0):
    U = np.zeros((NX,NT))
    U[:,0] = U0(x);
    U[0,:] = 0; U[-1,:] = 0;
    N = NX-1; gamma = (Alpha*dt)/(dx**2)
    A = np.diagflat((1+2*gamma)*np.ones(N-1),k=0) \
    +np.diagflat(-gamma*np.ones(N-2),k=1)+np.diagflat(-gamma*np.ones(N-2),k=-1)
    Ainv = np.linalg.inv(A)
    for n in range(0,NT-1):
        U[1:N,n+1] = np.dot(Ainv,U[1:N,n])
    return U
```

Résultats attendu

Résultats attendu



Sommaire

6 Simulation d'EDP Évolutive

- Simulation numérique de l'équation de transport
- Simulation Numérique de l'équation de la Chaleur
- Applications

Application 5

On considère le modèle suivant :

$$\begin{cases} \frac{\partial u}{\partial t} + \alpha \frac{\partial u}{\partial x} = f & \text{dans }]0, 10] \times [0, T_f] \\ u(0, t) = g(t) = x + 1 & \text{sur } [0, T_f] \\ u(x, 0) = \begin{cases} 1 & \text{dans }]0, 5[\\ 0 & \text{dans } [0, 10[\end{cases} \end{cases} \quad (11)$$

- 1) Simuler ce cas en utilisant les schémas décentré gauche et de Lax-Wendroff.
- 2) Comparer en terme de vitesse de convergence et d'ordre de convergence ces deux schémas. (On comparera les zones asymptotiques de convergence et les ordres de convergence suivant plusieurs normes.
- 3) Quelles conclusions faites-vous par rapport à la régularité de la solution initiale $u_0(x)$.

Application 6

Soit l'équation de diffusion en $2D$ suivant avec $\Omega = [0, L_x] \times [0, L_y]$:

$$\begin{cases} \frac{\partial u}{\partial t} - \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = f(x, y) & \text{dans } \Omega \times [0, T_f] \\ u(x, y, t) = 0 & \text{sur } \partial\Omega \times [0, T_f] \\ u(x, y, 0) = u_0(x, y) & \text{dans } \Omega \end{cases} \quad (12)$$

Il s'agit de simuler ce système avec :

- le terme source : $f(x, y) = \left(\left(\frac{2\pi}{L_x} \right)^2 + \left(\frac{2\pi}{L_y} \right)^2 \right) \sin \left(\frac{2\pi x}{L_x} \right) \cos \left(\frac{2\pi y}{L_y} \right)$,
- la condition initiale : $u_0(x, y) = 0$
- la solution exacte : $u(x, y, t \rightarrow \infty) = \sin \left(\frac{2\pi x}{L_x} \right) \cos \left(\frac{2\pi y}{L_y} \right)$

Le but est de résoudre numériquement ce problème avec un schéma de différence finie, explicite. La solution recherchée est une solution convergée en temps, telle que $\|U^{n+1} - U^n\| \leq 10^{-8}$. Cette solution sera comparée à la solution exacte donnée. Dans un premier temps ce schéma :

$$\frac{U_{i,j}^{n+1} - U_{i,j}^n}{\Delta t} + \frac{2U_{i,j}^n - U_{i-1,j}^n - U_{i+1,j}^n}{\Delta x^2} + \frac{2U_{i,j}^n - U_{i,j-1}^n - U_{i,j+1}^n}{\Delta y^2}$$

sera utilisé en écriture matricielle, ensuite une condition d'arrêt sera mise sur la boucle en temps pour regarder la convergence de la solution. Enfin une comparaison de la solution convergée avec celle analytique sera faite.